Walchand Institute of Technology Solapur

A MINI PROJECT REPORT ON

CLASSICRYPT

An Interactive Classical Cryptography Toolkit

NETWORK SECURITY

Student Name: Mayuri M. Gaikwad

Roll Number: 49

Class: Final Year ECM

Department: Electronic and Computer Engineering

Guide Name: Dr. Mrs. Rupali Shelke Mam

Academic Year: 2026–27

1. Abstract

ClassiCrypt is a web-based educational toolkit that demonstrates the working of classical cryptographic algorithms. Cryptography is the practice of transforming readable information, known as plaintext, into an unreadable form called ciphertext, and back again, using a process of encryption and decryption. This project focuses specifically on classical ciphers — substitution and transposition techniques that were used before the advent of modern computer-based cryptography. The toolkit allows a user to select an algorithm, provide input text and a key, and observe both the resulting ciphertext or plaintext and a step-by-step breakdown of how the algorithm produced it. The project is built entirely using HTML, CSS, and vanilla JavaScript, with no backend server, database, or external framework, and is deployed as a static site using GitHub Pages. Its primary purpose is educational: to help students visualize and understand how classical encryption and decryption techniques work internally, rather than to provide cryptographic security for real-world sensitive communication.

2. Introduction

Cryptography is the science of protecting information by transforming it into a form that cannot be easily understood by unauthorized parties. The original, readable message is called plaintext, and the transformed, unreadable message is called ciphertext. The process of converting plaintext into ciphertext is known as encryption, while the reverse process of recovering the original plaintext from ciphertext is known as decryption. Both processes typically rely on a key, a piece of secret information that controls how the transformation is performed.

Classical cryptography refers to encryption techniques that were developed and used before the modern computer era, including substitution ciphers (which replace letters with other letters or symbols) and transposition ciphers (which rearrange the order of letters without changing them). While these techniques are no longer secure against modern computational attacks, they remain valuable for teaching the fundamental principles on which all cryptography — including modern algorithms — is built. The ClassiCrypt project applies these concepts directly: it implements eight classical ciphers and allows a user to interactively encrypt and decrypt text, observe the underlying transformation, and compare how different algorithms behave on the same input.

3. Objectives

- To implement classical cryptographic algorithms — both substitution-based and transposition-based — using vanilla JavaScript.
- To provide an interactive web interface where a user can select a cipher, enter text and a key, and perform encryption or decryption.
- To visually demonstrate, step by step, how each cipher transforms plaintext into ciphertext and vice versa.
- To demonstrate the complete cryptographic chain — from plaintext to ciphertext and back to plaintext — within a single guided workflow.

- To allow independent testing of decryption, so a user can verify decryption logic without first performing encryption.
- To build and deploy the toolkit as a lightweight, framework-free, publicly accessible static website.

4. Technology Stack

The project uses only front-end web technologies, with no server-side component:

- HTML — defines the structure of the web pages, including the encryption/decryption workspace, algorithm reference, and help sections.
- CSS — provides the visual styling and a responsive, mobile-friendly layout.
- JavaScript (Vanilla) — implements all cipher algorithms, handles user interaction, and dynamically updates the interface; no external JavaScript framework is used.
- GitHub — used for version control and source code hosting.
- GitHub Pages — used to deploy the completed project as a live, publicly accessible static website.

5. System Working

At a high level, the toolkit follows this basic flow for any operation:

User → Web Interface → Select Cipher Algorithm → Enter Text and Key → JavaScript Algorithm Logic → Encryption / Decryption → Result Display

The user selects a cipher from the available list, enters the text to be processed and a suitable key, and the corresponding JavaScript function performs the transformation entirely within the browser. The result, along with the time taken to compute it, is displayed immediately.

In addition to independent encryption and decryption, ClassiCrypt implements a complete learning chain that mirrors the full cryptographic cycle:

Plaintext → Encryption → Ciphertext → Automatically transferred to the Decryption input → User clicks Decrypt → Original Plaintext

When a user encrypts a piece of text, the resulting ciphertext is automatically carried over into the decryption input, along with the same cipher selection and key. Decryption, however, is never performed automatically — the user must explicitly click the Decrypt action to complete the cycle. This reinforces, in a hands-on manner, that encryption and decryption are inverse operations of one another.

Independent decryption is also fully supported: a user may directly enter a ciphertext and a key, without first performing an encryption, and obtain the corresponding plaintext. This allows the decryption logic of each algorithm to be tested and understood on its own.

6. Implemented Algorithms and Features

The following eight classical cipher algorithms are implemented in the project:

Cipher	Category	Description
Caesar Cipher	Substitution	Shifts every letter by a fixed numeric key.
Monoalphabetic Cipher	Substitution	Maps each letter to a unique substitute letter using a 26-letter key.
Playfair Cipher	Substitution	Encrypts letters in pairs using a 5×5 key-square grid.
Hill Cipher	Substitution	Uses 2×2 matrix multiplication (mod 26) on letter pairs.
Vigenère Cipher	Polyalphabetic Substitution	Shifts letters using a repeating keyword, varying the shift per letter.
One-Time Pad	Substitution	Combines plaintext with a random key of equal length for each letter.
Rail Fence Cipher	Transposition	Rearranges letters in a zig-zag pattern across a chosen number of rails.
Columnar Transposition	Transposition	Writes text into a grid and reads columns in keyword-defined order.

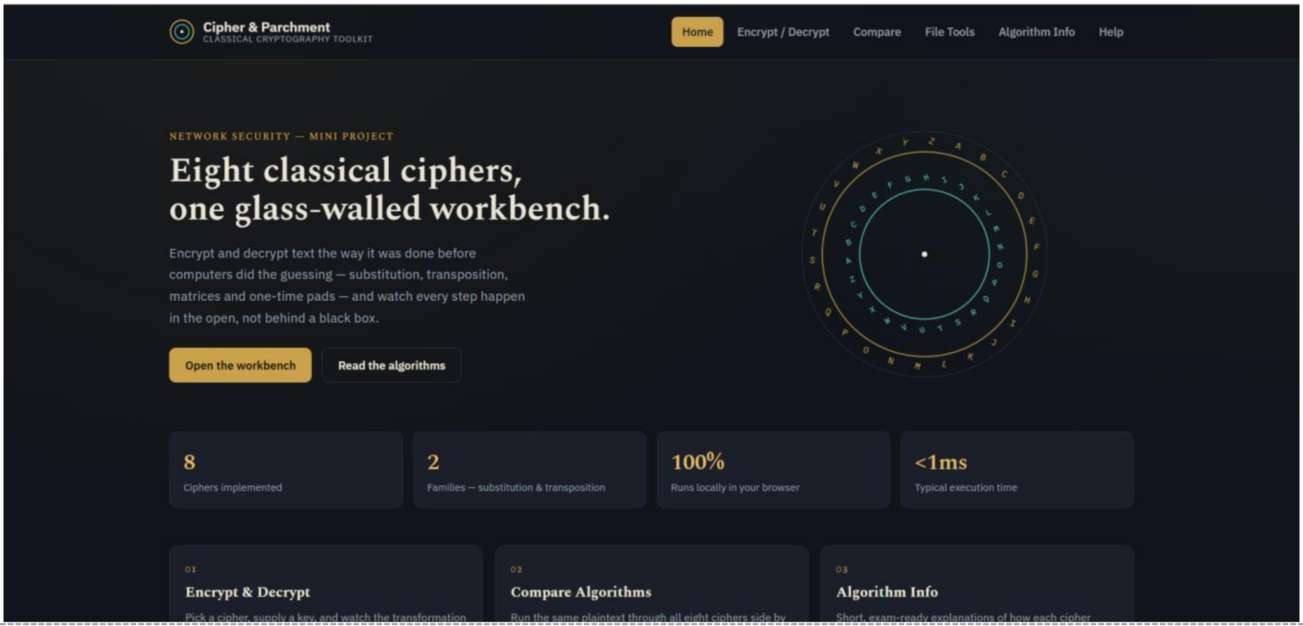
In addition to the algorithms above, the toolkit implements the following features:

- Encryption and Decryption of user-provided text for each of the eight ciphers.
- Automatic encryption-to-decryption flow, carrying ciphertext, cipher selection, and key from the encryption step into the decryption step.
- Independent decryption, allowing direct testing of the decryption process.
- Step-by-step process visualization, showing a character-by-character or block-by-block breakdown of how each algorithm computed its result.
- Compare Algorithms module, which runs the same plaintext through all eight ciphers simultaneously and displays the resulting ciphertext and execution time for each.
- File encryption and decryption, allowing a user to upload a text file, process it with a chosen cipher, and download the result.
- Algorithm Information section, providing a short description, key format, and example for each cipher.
- Help section, offering usage instructions and answers to frequently asked questions.
- A password-protected access screen that requires the user to enter a password before the toolkit interface becomes accessible.

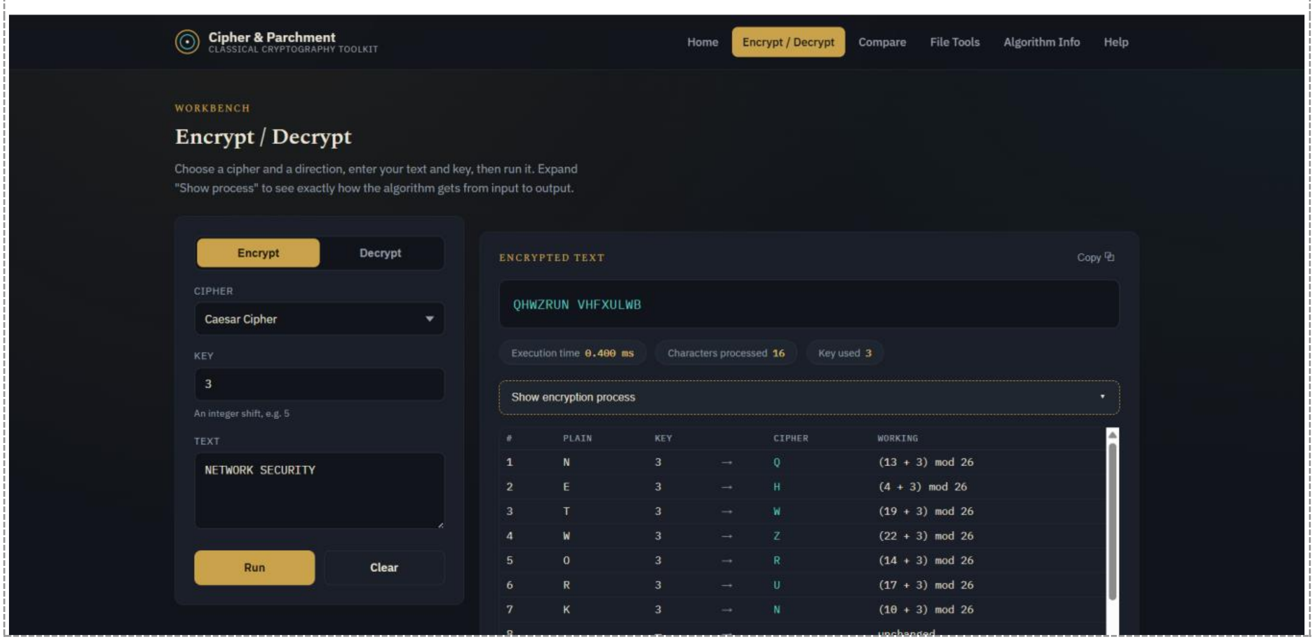
7. Results and Screenshots

The completed ClassiCrypt toolkit successfully performs encryption and decryption for all eight implemented classical ciphers, correctly reproducing the original plaintext when a valid key is used for decryption. The step-by-step process visualization was verified to accurately reflect the internal working of each algorithm, and the automatic encryption-to-decryption flow was confirmed to correctly carry the ciphertext, cipher choice, and key between the two steps without performing decryption automatically. The following screenshots illustrate the working system:

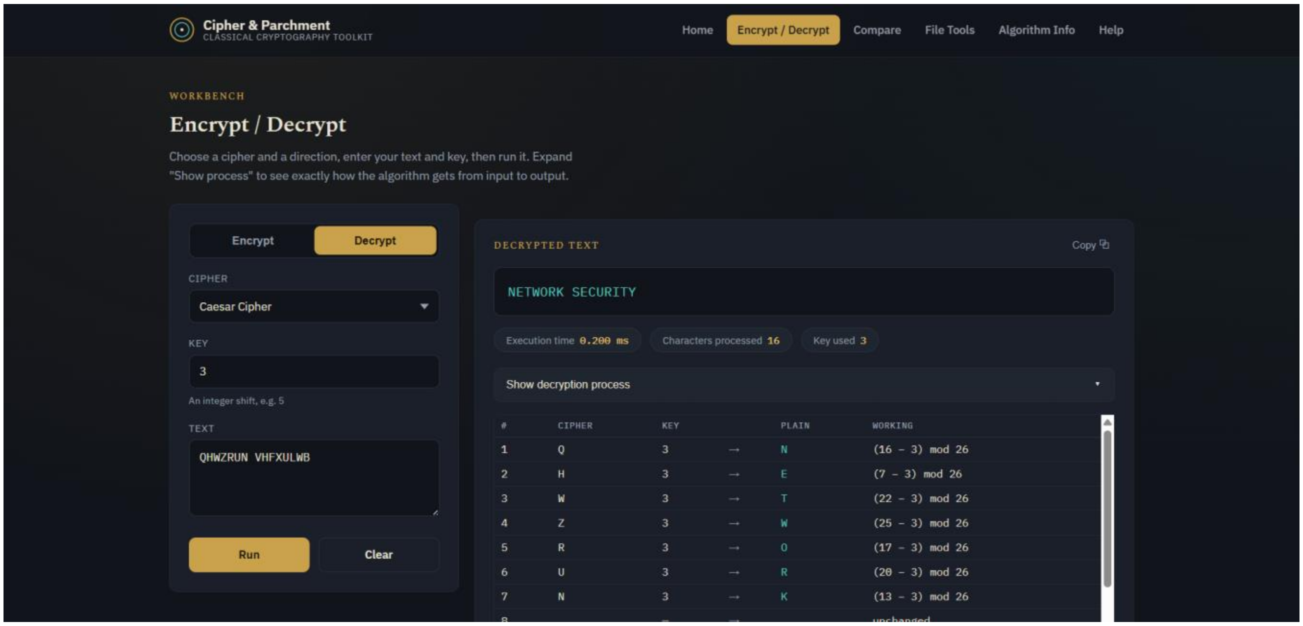
HOME PAGE



ENCRYPTION



DECRYPTION



8. Conclusion

ClassiCrypt successfully demonstrates the working of eight classical cryptographic algorithms through an interactive, browser-based toolkit built entirely with HTML, CSS, and vanilla JavaScript. By allowing users to encrypt and decrypt text, observe the step-by-step transformation process, and follow the complete plaintext-to-ciphertext-to-plaintext cycle, the project fulfils its educational purpose of making classical cryptographic concepts easier to understand and visualize. It reinforces core ideas such as plaintext, ciphertext, keys, substitution, and transposition through direct, hands-on interaction rather than theoretical description alone. It is important to note that the classical ciphers implemented in this project — while historically significant and valuable for learning — are not secure by modern standards and are not suitable for protecting real, sensitive communication. Overall, ClassiCrypt serves as an effective learning aid for understanding the foundational principles of cryptography.